

Data Platform Engineering Project

Lender Portfolio Intelligence Pipeline

Snowflake · dbt Core · Snowpipe · S3 · Cortex Analyst · GitHub Actions · Terraform

A production-grade portfolio project for TransUnion Canada

Author	Data Platform Engineer
Environment	Snowflake (trial) · dbt Core · S3 · GitHub Actions · Terraform
Estimated Duration	6 weeks
Difficulty	Intermediate — Advanced
Key Skills Demonstrated	IaC · CI/CD · ELT · Data Modelling · Cortex AI · RBAC
Target Role	Data Platform Engineer — Financial Services, Toronto

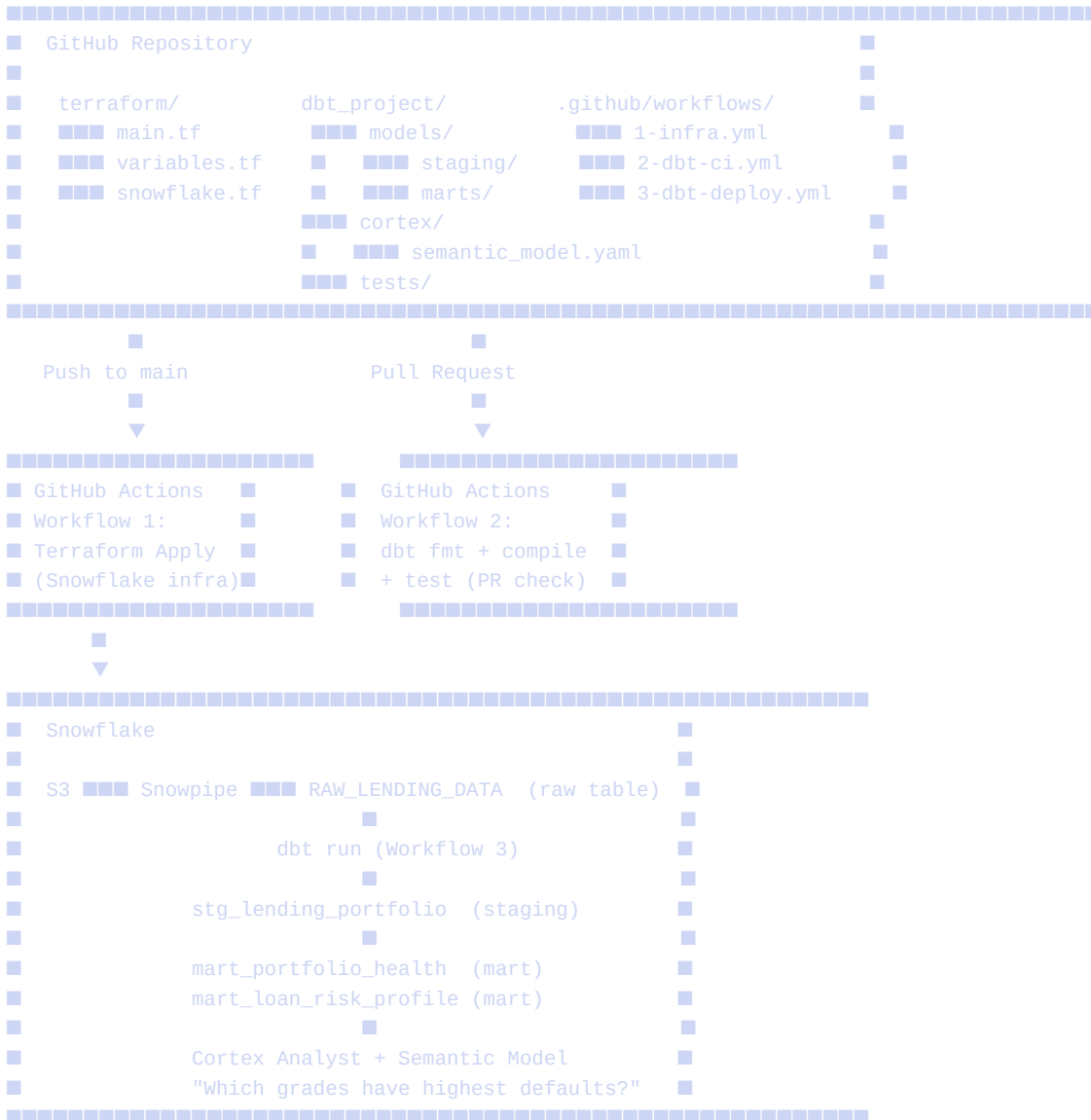
1. Why This Project Is Interview-Ready

This is not a tutorial project. It combines four enterprise-grade disciplines into a single coherent pipeline — the same combination that data platform teams at Canadian banks, pension funds, and fintechs are actively building right now:

Discipline	What It Demonstrates	Relevant To
Infrastructure as Code	Terraform provisions Snowflake objects	All financial employers
CI/CD Pipelines	GitHub Actions deploys infra and dbt models	Banks, fintechs, pension funds
ELT + Data Modelling	dbt staging, mart, tests, lineage docs	Data platform teams everywhere
Snowflake Cortex AI	Natural language Q&A on modelled financial data	Firms evaluating Cortex Analyst

Cortex Analyst is Snowflake's built-in natural language to SQL engine. You define a semantic model (a YAML file) on top of your dbt mart tables, and business users can ask questions in plain English — 'Which loan grades have the highest default rate?' — and Cortex generates and executes the SQL automatically. Financial services firms are actively piloting this for internal analyst self-service. Building it in a portfolio project places you ahead of most candidates who haven't touched it yet.

2. End-to-End Architecture



3. Dataset

Use the **Lending Club Loan Data** from Kaggle. It contains over 2 million consumer loan records with rich attributes across credit grade, loan purpose, employment, income, delinquency history, and interest rates. This is the best publicly available dataset for a TransUnion-style credit portfolio scenario — it has enough dimensions to make Cortex Analyst genuinely interesting.

Dataset URL: <https://www.kaggle.com/datasets/wordsforthewise/lending-club> — Download **accepted_2007_to_2018Q4.csv.gz** (2.26 GB, ~2.2M rows). Use a 100K-row sample for development.

Key columns you will work with

Column	Type	Description	Used In
id	VARCHAR	Unique loan identifier	Staging key
loan_amnt	FLOAT	Requested loan amount	Mart metrics
funded_amnt	FLOAT	Amount actually funded	Mart metrics
int_rate	FLOAT	Interest rate on the loan	Risk mart
grade	VARCHAR	LC assigned loan grade (A-G)	Risk mart + Cortex
sub_grade	VARCHAR	Sub-grade (A1-G5)	Risk mart
emp_length	VARCHAR	Employment length in years	Staging clean
home_ownership	VARCHAR	RENT / OWN / MORTGAGE	Mart + Cortex
annual_inc	FLOAT	Borrower annual income	Risk mart
loan_status	VARCHAR	Current/Charged Off/Default/etc.	Mart + Cortex
purpose	VARCHAR	Loan purpose (debt_consolidation etc)	Mart + Cortex
dti	FLOAT	Debt-to-income ratio	Risk mart
delinq_2yrs	NUMBER	Delinquencies in past 2 years	Risk mart
open_acc	NUMBER	Open credit lines	Risk mart
issue_d	DATE	Loan issue date	Time dimension
addr_state	VARCHAR	Borrower state	Cortex geography

Sample Cortex Analyst questions this dataset enables

Once your dbt marts are built and the semantic model is defined, Cortex Analyst can answer questions like these in natural language:

- Which loan grades have the highest default rate?
- What is the average interest rate for debt consolidation loans vs home improvement?
- Which states have the highest debt-to-income ratio on average?
- How has the volume of charged-off loans changed year over year?
- What is the average loan amount for borrowers who own vs rent their home?

- *Which employment length bucket has the lowest delinquency rate?*

4. Repository Structure

```
lender-portfolio-pipeline/
├── .github/
│   ├── workflows/
│   │   ├── 1-infra-deploy.yml    # Terraform: provision Snowflake objects
│   │   ├── 2-dbt-ci.yml         # PR check: fmt + compile + test
│   │   └── 3-dbt-deploy.yml      # Push to main: dbt run + test
│   └──
├── terraform/
│   ├── main.tf                  # Snowflake provider
│   ├── variables.tf
│   ├── snowflake_objects.tf     # DB, schema, warehouse, stage, pipe
│   └── rbac.tf                  # Roles and grants
├── dbt_project/
│   ├── dbt_project.yml
│   ├── packages.yml
│   ├── models/
│   │   ├── staging/
│   │   │   ├── sources.yml
│   │   │   └── stg_lending_portfolio.sql
│   │   ├── marts/
│   │   │   ├── schema.yml
│   │   │   ├── mart_portfolio_health.sql
│   │   │   └── mart_loan_risk_profile.sql
│   ├── cortex/
│   │   ├── semantic_model.yaml  # Cortex Analyst semantic model
│   └── tests/
│       └── assert_no_negative_income.sql
├── snowflake/
│   └── bootstrap.sql            # Admin: user, role, account grants
├── .gitignore
└── README.md
```

.gitignore must include: profiles.yml | *.tfstate | *.tfstate.backup | .terraform/ | target/ | dbt_packages/ | logs/ | .env

5. Phase 1 — Terraform: Snowflake Infrastructure (Week 1)

Provision all Snowflake objects with Terraform. This reuses the pattern from your existing Project 3 — same provider, same structure, extended for this pipeline.

terraform/snowflake_objects.tf

```
resource "snowflake_database" "lending_db" {
  name      = "LENDING_PIPELINE_DB"
  comment   = "Lender portfolio intelligence pipeline"
}

resource "snowflake_schema" "raw" {
  database = snowflake_database.lending_db.name
  name     = "RAW"
}

resource "snowflake_schema" "staging" {
  database = snowflake_database.lending_db.name
  name     = "STAGING"
}

resource "snowflake_schema" "marts" {
  database = snowflake_database.lending_db.name
  name     = "MARTS"
}

resource "snowflake_schema" "cortex" {
  database = snowflake_database.lending_db.name
  name     = "CORTEX"
}

resource "snowflake_warehouse" "pipeline_wh" {
  name                = "PIPELINE_WH"
  warehouse_size      = "XSMALL"
  auto_suspend        = 60
  initially_suspended = true
}

resource "snowflake_table" "raw_lending" {
  database = snowflake_database.lending_db.name
  schema   = snowflake_schema.raw.name
  name     = "RAW_LENDING_DATA"

  column { name = "id";          type = "VARCHAR" }
  column { name = "loan_amnt";   type = "FLOAT" }
  column { name = "funded_amnt"; type = "FLOAT" }
  column { name = "int_rate";    type = "FLOAT" }
  column { name = "grade";       type = "VARCHAR" }
  column { name = "sub_grade";   type = "VARCHAR" }
  column { name = "emp_length";  type = "VARCHAR" }
  column { name = "home_ownership"; type = "VARCHAR" }
  column { name = "annual_inc";  type = "FLOAT" }
  column { name = "loan_status"; type = "VARCHAR" }
  column { name = "purpose";     type = "VARCHAR" }
  column { name = "dti";         type = "FLOAT" }
  column { name = "delinq_2yrs"; type = "NUMBER" }
  column { name = "open_acc";    type = "NUMBER" }
  column { name = "issue_d";     type = "VARCHAR" }
  column { name = "addr_state";  type = "VARCHAR" }
```

```
    column { name = "_loaded_at";    type = "TIMESTAMP_NTZ";  
            default { constant = "CURRENT_TIMESTAMP()" } }  
}
```

terraform/rbac.tf

```
resource "snowflake_grant_privileges_to_account_role" "transformer_db" {  
  account_role_name = var.transformer_role  
  privileges        = ["USAGE"]  
  on_account_object {  
    object_type = "DATABASE"  
    object_name = snowflake_database.lending_db.name  
  }  
}  
  
resource "snowflake_grant_privileges_to_account_role" "transformer_wh" {  
  account_role_name = var.transformer_role  
  privileges        = ["USAGE", "OPERATE"]  
  on_account_object {  
    object_type = "WAREHOUSE"  
    object_name = snowflake_warehouse.pipeline_wh.name  
  }  
}
```


6. GitHub Actions — Workflow 1: Infrastructure Deploy

This workflow provisions or updates Snowflake infrastructure via Terraform whenever changes are pushed to the **terraform/** directory on main. A manual apply gate prevents accidental changes.

.github/workflows/1-infra-deploy.yml

```
name: Snowflake Infrastructure Deploy

on:
  push:
    branches: [main]
    paths: ["terraform/**"]
  workflow_dispatch:
    inputs:
      apply:
        description: "Run terraform apply?"
        type: choice
        options: ["false", "true"]
        default: "false"

jobs:
  terraform:
    runs-on: ubuntu-latest
    environment: snowflake-dev

    env:
      TF_VAR_snowflake_organization_name: ${ secrets.SNOWFLAKE_ORGANIZATION }
      TF_VAR_snowflake_account_name:      ${ secrets.SNOWFLAKE_ACCOUNT }
      TF_VAR_snowflake_user:              ${ secrets.SNOWFLAKE_USER }
      TF_VAR_snowflake_password:          ${ secrets.SNOWFLAKE_PASSWORD }
      TF_VAR_snowflake_role:              ${ secrets.SNOWFLAKE_ROLE }
      TF_VAR_transformer_role:            ${ secrets.TRANSFORMER_ROLE }

    steps:
      - uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3

      - name: Terraform Format Check
        run: terraform fmt -check -recursive
        working-directory: terraform

      - name: Terraform Init
        run: terraform init
        working-directory: terraform

      - name: Terraform Validate
        run: terraform validate
        working-directory: terraform

      - name: Terraform Plan
        run: terraform plan -out=tfplan
        working-directory: terraform

      - name: Upload plan artifact
```


7. GitHub Actions — Workflows 2 & 3: dbt CI/CD

Workflow 2 — PR Check (.github/workflows/2-dbt-ci.yml)

Runs on every pull request targeting main. Catches formatting and compilation errors before they reach the main branch.

```
name: dbt CI — PR Check

on:
  pull_request:
    branches: [main]
    paths: ["dbt_project/**"]

jobs:
  dbt-ci:
    runs-on: ubuntu-latest
    environment: snowflake-dev

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }

      - name: Install dbt
        run: pip install dbt-snowflake

      - name: dbt deps
        run: dbt deps
        working-directory: dbt_project

      - name: dbt format check
        run: dbt format --check
        working-directory: dbt_project

      - name: dbt compile
        env:
          SNOWFLAKE_USER:      ${ secrets.SNOWFLAKE_USER }
          SNOWFLAKE_PASSWORD:  ${ secrets.SNOWFLAKE_PASSWORD }
          SNOWFLAKE_ACCOUNT:   ${ secrets.SNOWFLAKE_ACCOUNT }
          SNOWFLAKE_ROLE:      ${ secrets.TRANSFORMER_ROLE }
        run: dbt compile
        working-directory: dbt_project
```

Workflow 3 — Deploy (.github/workflows/3-dbt-deploy.yml)

Runs on push to main after PR is merged. Executes the full dbt build (run + test) in the Snowflake environment.

```
name: dbt Deploy — Run and Test

on:
  push:
    branches: [main]
    paths: ["dbt_project/**"]
    workflow_dispatch: {}

jobs:
```

```
dbt-deploy:
  runs-on: ubuntu-latest
  environment: snowflake-dev

  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-python@v5
      with: { python-version: '3.11' }

    - name: Install dbt
      run: pip install dbt-snowflake

    - name: dbt deps
      run: dbt deps
      working-directory: dbt_project

    - name: dbt build (run + test)
      env:
        SNOWFLAKE_USER:      ${ secrets.SNOWFLAKE_USER }
        SNOWFLAKE_PASSWORD:  ${ secrets.SNOWFLAKE_PASSWORD }
        SNOWFLAKE_ACCOUNT:   ${ secrets.SNOWFLAKE_ACCOUNT }
        SNOWFLAKE_ROLE:      ${ secrets.TRANSFORMER_ROLE }
      run: dbt build --target prod
      working-directory: dbt_project

    - name: Upload dbt artifacts
      uses: actions/upload-artifact@v4
      with:
        name: dbt-artifacts
        path: dbt_project/target/
```

GitHub Repository Secrets required: SNOWFLAKE_ORGANIZATION | SNOWFLAKE_ACCOUNT | SNOWFLAKE_USER | SNOWFLAKE_PASSWORD | SNOWFLAKE_ROLE | TRANSFORMER_ROLE

8. Phase 2 — Snowpipe & S3 Ingestion (Week 2)

Run this SQL after Terraform has provisioned the Snowflake objects. The storage integration and Snowpipe are created manually (not in Terraform) because they require the IAM trust relationship to be configured first.

```
-- Step 1: Create storage integration
CREATE STORAGE INTEGRATION s3_lending_integration
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_AWS_ROLE_ARN = '<your_iam_role_arn>'
  STORAGE_ALLOWED_LOCATIONS = ('s3://<your-bucket>/lending-data/');

-- Step 2: Get AWS_IAM_USER_ARN and EXTERNAL_ID for your IAM trust policy
DESC INTEGRATION s3_lending_integration;

-- Step 3: Create external stage
CREATE OR REPLACE STAGE LENDING_PIPELINE_DB.RAW.LENDING_S3_STAGE
  STORAGE_INTEGRATION = s3_lending_integration
  URL = 's3://<your-bucket>/lending-data/'
  FILE_FORMAT = (
    TYPE = CSV
    FIELD_OPTIONALLY_ENCLOSED_BY = ''
    SKIP_HEADER = 1
    NULL_IF = ('NULL', 'null', 'NA', '')
  );

-- Step 4: Create Snowpipe
CREATE OR REPLACE PIPE LENDING_PIPELINE_DB.RAW.LENDING_PIPE
  AUTO_INGEST = TRUE
AS
COPY INTO LENDING_PIPELINE_DB.RAW.RAW_LENDING_DATA (
  id, loan_amnt, funded_amnt, int_rate, grade, sub_grade,
  emp_length, home_ownership, annual_inc, loan_status,
  purpose, dti, delinq_2yrs, open_acc, issue_d, addr_state
)
FROM @LENDING_PIPELINE_DB.RAW.LENDING_S3_STAGE;

-- Step 5: Get the SQS ARN for S3 event notification
SHOW PIPES;
-- Copy notification_channel value → configure in S3 bucket Events

-- Validation: drop the CSV, wait 60s, then:
SELECT COUNT(*), MAX(_loaded_at) FROM LENDING_PIPELINE_DB.RAW.RAW_LENDING_DATA;
```

9. Phase 3 — dbt Staging Model (Week 3)

models/staging/sources.yml

```
version: 2
sources:
  - name: raw
    database: LENDING_PIPELINE_DB
    schema: RAW
    tables:
      - name: RAW_LENDING_DATA
        description: "Raw Lending Club data landed by Snowpipe from S3"
        columns:
          - name: id
            tests: [unique, not_null]
          - name: grade
            tests:
              - accepted_values:
                  values: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
          - name: loan_status
            tests: [not_null]
```

models/staging/stg_lending_portfolio.sql

```
WITH source AS (
  SELECT * FROM {{ source('raw', 'RAW_LENDING_DATA') }}
),
cleaned AS (
  SELECT
    id AS loan_id,
    UPPER(TRIM(grade)) AS loan_grade,
    UPPER(TRIM(sub_grade)) AS loan_sub_grade,
    UPPER(TRIM(loan_status)) AS loan_status,
    UPPER(TRIM(purpose)) AS loan_purpose,
    UPPER(TRIM(home_ownership)) AS home_ownership,
    UPPER(TRIM(addr_state)) AS borrower_state,

    -- Numeric casts
    CAST(loan_amnt AS DECIMAL(12,2)) AS loan_amount,
    CAST(funded_amnt AS DECIMAL(12,2)) AS funded_amount,
    CAST(annual_inc AS DECIMAL(14,2)) AS annual_income,
    ROUND(CAST(int_rate AS FLOAT), 4) AS interest_rate,
    ROUND(CAST(dti AS FLOAT), 4) AS debt_to_income_ratio,
    COALESCE(CAST(delinq_2yrs AS INT), 0) AS delinquencies_2yr,
    COALESCE(CAST(open_acc AS INT), 0) AS open_accounts,

    -- Parse employment length to numeric years
    CASE
      WHEN emp_length = '10+ years' THEN 10
      WHEN emp_length = '< 1 year' THEN 0
      WHEN emp_length IS NULL THEN NULL
      ELSE TRY_CAST(REGEXP_REPLACE(emp_length, '^[0-9]', '') AS INT)
    END AS employment_years,

    -- Parse issue date
    TRY_TO_DATE(issue_d, 'MON-YYYY') AS issue_date,

    -- Derived flags
```


10. Phase 3 — dbt Mart Models (Week 3 cont.)

models/marts/mart_portfolio_health.sql

```
WITH base AS (
  SELECT * FROM {{ ref('stg_lending_portfolio') }}
)
SELECT
  issue_date,
  loan_grade,
  loan_purpose,
  home_ownership,

  COUNT(*) AS total_loans,
  SUM(loan_amount) AS total_loan_volume,
  ROUND(AVG(loan_amount), 2) AS avg_loan_amount,
  ROUND(AVG(interest_rate) * 100, 2) AS avg_interest_rate_pct,
  ROUND(AVG(debt_to_income_ratio), 4) AS avg_dti,
  ROUND(AVG(annual_income), 2) AS avg_annual_income,

  SUM(CASE WHEN is_defaulted THEN 1 ELSE 0 END) AS defaulted_loans,
  ROUND(
    SUM(CASE WHEN is_defaulted THEN 1 ELSE 0 END)
    / NULLIF(COUNT(*), 0) * 100, 2
  ) AS default_rate_pct,

  CURRENT_TIMESTAMP() AS mart_updated_at
FROM base
WHERE issue_date IS NOT NULL
GROUP BY 1,2,3,4
```

models/marts/mart_loan_risk_profile.sql

```
WITH base AS (
  SELECT * FROM {{ ref('stg_lending_portfolio') }}
)
SELECT
  loan_grade,
  loan_sub_grade,
  borrower_state,
  employment_years,

  COUNT(*) AS loan_count,
  ROUND(AVG(interest_rate) * 100, 2) AS avg_interest_rate_pct,
  ROUND(AVG(delinquencies_2yr), 2) AS avg_delinquencies,
  ROUND(AVG(open_accounts), 1) AS avg_open_accounts,
  ROUND(AVG(debt_to_income_ratio), 4) AS avg_dti,

  SUM(CASE WHEN is_defaulted THEN 1 ELSE 0 END) AS defaults,
  ROUND(
    SUM(CASE WHEN is_defaulted THEN 1 ELSE 0 END)
    / NULLIF(COUNT(*), 0) * 100, 2
  ) AS default_rate_pct,

  CURRENT_TIMESTAMP() AS mart_updated_at
FROM base
GROUP BY 1,2,3,4
```


Run and test all models

```
# Run the full pipeline  
dbt build
```

```
# Or step by step
```

```
dbt run --select stg_lending_portfolio mart_portfolio_health mart_loan_risk_profile  
dbt test --select stg_lending_portfolio mart_portfolio_health mart_loan_risk_profile
```

```
# Generate and view lineage docs
```

```
dbt docs generate && dbt docs serve
```

11. Phase 4 — Snowflake Cortex Analyst (Week 4–5)

Cortex Analyst lets business users ask questions about your mart tables in plain English. You define a **semantic model** — a YAML file that describes your tables, columns, metrics, and relationships in business terms. Cortex uses this to generate accurate SQL without the user needing to know the schema.

What Cortex Analyst is NOT: It is not a chatbot on top of raw data. It requires clean, well-modelled data — which is exactly what your dbt marts provide. This is why the dbt work must come first. Your staging and mart models are the foundation that makes Cortex reliable.

dbt_project/cortex/semantic_model.yaml

```
name: lending_portfolio_semantic_model
description: >
  Semantic model for the Lending Club portfolio health and risk marts.
  Enables natural language analysis of loan performance, default rates,
  and borrower risk profiles.

tables:
- name: mart_portfolio_health
  description: "Monthly portfolio health by loan grade and purpose"
  base_table:
    database: LENDING_PIPELINE_DB
    schema: MARTS
    table: MART_PORTFOLIO_HEALTH

dimensions:
- name: loan_grade
  description: "Lending Club assigned grade A (lowest risk) to G (highest risk)"
  expr: loan_grade
  data_type: TEXT
  sample_values: ['A', 'B', 'C', 'D', 'E', 'F', 'G']

- name: loan_purpose
  description: "Purpose of the loan e.g. debt_consolidation, home_improvement"
  expr: loan_purpose
  data_type: TEXT

- name: home_ownership
  description: "Borrower home ownership: RENT, OWN, MORTGAGE"
  expr: home_ownership
  data_type: TEXT

- name: issue_date
  description: "Date the loan was issued"
  expr: issue_date
  data_type: DATE

measures:
- name: total_loans
  description: "Total number of loans"
  expr: total_loans
  data_type: NUMBER
  default_aggregation: sum

- name: total_loan_volume
  description: "Total dollar value of loans originated"
```


11 (cont.) — Uploading the Semantic Model & Querying

Step 1 — Upload semantic model to Snowflake stage

```
-- Create a stage for the semantic model YAML
CREATE STAGE IF NOT EXISTS LENDING_PIPELINE_DB.CORTEX.SEMANTIC_MODELS
  DIRECTORY = (ENABLE = TRUE);

-- Upload from local (run in SnowSQL or VS Code Snowflake extension)
PUT file://dbt_project/cortex/semantic_model.yaml
  @LENDING_PIPELINE_DB.CORTEX.SEMANTIC_MODELS
  AUTO_COMPRESS = FALSE
  OVERWRITE = TRUE;

-- Verify it uploaded
LIST @LENDING_PIPELINE_DB.CORTEX.SEMANTIC_MODELS;
```

Step 2 — Query using Cortex Analyst REST API

Cortex Analyst is accessed via a REST API call. You can call it from Python, from a Streamlit app, or directly from the Snowflake UI:

```
import snowflake.connector
import json

conn = snowflake.connector.connect(
    user=      "<your_user>",
    password=  "<your_password>",
    account=   "<org>-<account>",
    warehouse="PIPELINE_WH",
    database=  "LENDING_PIPELINE_DB",
    role=      "TRANSFORMER_ROLE"
)

question = "Which loan grades have the highest default rate?"

sql = (
    "SELECT SNOWFLAKE.CORTEX.COMPLETE("
    "  'mistral-large2', "
    "  ARRAY_CONSTRUCT("
    "    OBJECT_CONSTRUCT('role', 'user', 'content', %s)"
    "  )"
    ")"
)

response = conn.cursor().execute(sql, (question,)).fetchone()

print(json.loads(response[0])['choices'][0]['messages'])
```

Step 3 — Sample Cortex Analyst questions to validate

Q: Which loan grades have the highest default rate?

Expected SQL shape: SELECT loan_grade, AVG(default_rate_pct) ... ORDER BY 2 DESC

Q: What is the average interest rate for debt consolidation vs home improvement?

Expected SQL shape: SELECT loan_purpose, AVG(avg_interest_rate_pct) ... WHERE loan_purpose IN (...)

Q: Which states have the highest average DTI?

Expected SQL shape: SELECT borrower_state, AVG(avg_dti) ... ORDER BY 2 DESC LIMIT 10

Q: *How has the total loan volume changed year over year?*

Expected SQL shape: SELECT YEAR(issue_date), SUM(total_loan_volume) ... GROUP BY 1 ORDER BY 1

12. Week-by-Week Execution Plan

Week	Focus	Key Deliverable	Done?
1	Terraform + GitHub Actions	Snowflake objects provisioned by Workflow 1 from GitHub	■
2	Snowpipe + S3 ingestion	CSV lands in S3, raw table auto-populated via Snowpipe	■
3	dbt staging + mart models	Both marts built, all tests green, dbt build passes	■
4	dbt CI/CD workflows	Workflow 2 (PR check) and Workflow 3 (deploy) both green	■
5	Cortex semantic model	Semantic model uploaded, 4 natural language Q&As working	■
6	Polish + documentation	README with architecture diagram, dbt docs live, LinkedIn post	■

13. GitHub Repository Secrets

Secret Name	Used By	Value
SNOWFLAKE_ORGANIZATION	Terraform + dbt	Your Snowflake org name
SNOWFLAKE_ACCOUNT	Terraform + dbt	Your Snowflake account name
SNOWFLAKE_USER	Terraform + dbt	TERRAFORM_USER / service account
SNOWFLAKE_PASSWORD	Terraform + dbt	Stored as GitHub encrypted secret
SNOWFLAKE_ROLE	Terraform	TERRAFORM_ROLE
TRANSFORMER_ROLE	dbt workflows	TRANSFORMER_ROLE (dbt user role)

Environment: Create a GitHub Environment named **snowflake-dev** and add all secrets there. This adds an approval gate before any workflow can run against your Snowflake account — an enterprise pattern worth demonstrating.

14. Interview Talking Points

These are the questions a hiring manager at a bank, pension fund, or fintech will realistically ask about this project. Answer from direct experience.

Q: Walk me through your CI/CD pipeline.

Three GitHub Actions workflows: Workflow 1 provisions Snowflake objects via Terraform when infra files change on main. Workflow 2 runs on every PR — it compiles and checks dbt format. Workflow 3 runs dbt build on merge to main. All secrets are stored in a GitHub Environment with an approval gate, which reflects how this is managed in regulated environments.

Q: Why Terraform for Snowflake objects? Why not just run SQL?

SQL scripts are imperative and stateful — if someone manually changes a warehouse size, there is no record and no way to detect drift. Terraform is declarative — the state file tracks what was deployed, plan shows drift, and apply reconciles it. That repeatability is what you need for dev/prod parity.

Q: What does your dbt staging layer actually do?

Three things only: rename columns to snake_case, cast data types correctly, and filter out records that would fail downstream constraints. No business logic. That separation means the mart layer is independently testable and the staging layer can be reused by multiple marts.

Q: What is Cortex Analyst and why did you use it?

Cortex is Snowflake's built-in LLM layer. Cortex Analyst lets you define a semantic model on top of dbt marts — column descriptions, measures, dimensions — and then business users can ask questions in plain English. The semantic model is what makes it accurate; without the dbt work underneath, the answers would be unreliable. Financial firms are actively evaluating this for analyst self-service.

Q: How do you handle bad data from the source file?

Two layers: the Snowpipe COPY command uses NULL_IF to handle common null representations. Then the dbt staging model applies WHERE filters — removing null IDs, invalid grades, zero loan amounts — before any mart model touches the data. dbt tests on the staging layer catch anything that still slips through.

Q: What would you change if this were production?

Remote Terraform state in S3 with state locking. dbt incremental models instead of full refresh for large files. Separate dev and prod Snowflake databases with dbt targets. A scheduled orchestration layer — Airflow or Prefect — instead of relying on GitHub Actions timing. Alerting on dbt test failures.

15. Future Enhancements

Enhancement	What It Adds
Incremental dbt models	Use <code>is_incremental()</code> macro so only new Snowpipe records are processed
Remote Terraform state	Store tfstate in S3 with DynamoDB locking — production standard
Airflow orchestration	Trigger dbt after Snowpipe completes rather than on a fixed schedule
dbt Snapshots	Track how loan statuses change over time using dbt snapshot tables
Streamlit on Snowflake	Build a Cortex Analyst UI directly in Snowflake using Streamlit in Snowflake
Dev/Prod environments	Separate Snowflake databases for dev and prod with dbt targets
Great Expectations	Add deeper profiling and expectation suites on top of dbt tests
Terraform Snowflake Tasks	Use Terraform to also manage Snowflake Tasks for scheduled refreshes

You are building what modern financial services data teams actually build.